Sakutto: On-Device Page Translation with a Small Instruction-Tuned Model

J. Ammaru

Independent researcher
sakutto@example.org

ABSTRACT

Machine translation of web pages is usually a client of a remote service. This paper describes Sakutto, a browser extension that translates a page entirely on the reader's own machine, and reports what moving a 0.8-billion-parameter instruction-tuned model into a browser tab costs and what it buys. The extension runs PLaMo 3 Translate through a WebAssembly build of llama.cpp, keeps the model weights in the browser's private file system, and translates a page block by block as the reader scrolls. On a four-core laptop with a discrete GPU the model reaches 34.8 tokens per second and finishes a 1,900-word article in 41 seconds; on the same machine's CPU it reaches 9.1 tokens per second. We describe the scheduling rules that keep the first paragraph visible within two seconds, the cache that makes a revisited page free, and the failure modes that a page collector has to handle before a reader will accept the result.

1. Introduction

A reader who wants an English article in Japanese has three choices: paste it into a translation website, install a browser extension that forwards the page to a server, or read the English. The first two send the page, and every selection the reader makes afterwards, to a third party. For a research paper under review, a private issue tracker or a medical record this is often unacceptable, and it is frequently prohibited by the terms the reader is bound by.

Running the model in the browser removes that transfer. The page text never leaves the process, no account is needed, and the extension works offline once the weights are downloaded. The cost is latency and memory: a transformer that a server would run on an accelerator has to fit in a tab, compete with the page for the GPU, and produce tokens one at a time in front of a reader who is already looking at the text.

This paper reports a working system rather than a new model. Section 2 describes the runtime, Section 3 the scheduling rules that decide what to translate first, and Section 4 the measurements. Section 5 discusses the failure modes we could not design away, and Section 6 lists the work that remains.

2. System Design

The extension is a Manifest V3 package with four processes. A service worker owns the lifetime of everything else. An offscreen document hosts the runtime, because a service worker cannot hold a WebAssembly heap across a suspension. A content script walks the page and writes translations back into the document. Popup, options and onboarding pages render the settings.

2.1 Runtime

The model is PLaMo 3 Translate, an 0.8-billion-parameter decoder trained for translation in 53 languages in either direction, quantised to four bits per weight. The published WebAssembly build of llama.cpp cannot load its tokenizer, so the extension ships its own build of the runtime at a pinned commit, patched in two places so that workers and the WebAssembly module load from packaged files rather than from blob URLs, which Manifest V3 forbids.

The runtime offers two backends. The WebGPU backend keeps two sequences resident and decodes a batch of up to eight short segments in a single completion. The CPU backend keeps one sequence and uses four to eight threads, depending on the machine. A probe runs at startup, and a failed GPU load falls back to the CPU path rather than failing the translation.

$$S_t = (I - \beta_t)\, S_{t-1}$$

> **Proposition 1.** A block whose translation does not fit its own box is left in the source typesetting rather than shown cut short.

```
1  for block in page:
2      if is_equation(block): continue
3      emit(translate(block))
```

2.2 Page model

A page is not a list of paragraphs. The collector therefore walks the document rather than selecting a fixed set of tags, and it treats four shapes differently: ordinary prose, an item of a grid or flex container, a cell of a repeating data row, and a container that owns text of its own beside its children. The distinction matters because a translation is written back into the text nodes the source already had. A row laid out as a grid becomes a vertical pile if its translation lands between the cells instead of inside them.

Inline elements that are not prose — code, keyboard keys, sample output, pronunciation and citation markers — stay in the document untouched and become numbered placeholders in the text the model sees. The model is asked to keep each placeholder where the sentence had it, and a translation that moves or drops one is rejected and retried without placeholders.

41 s

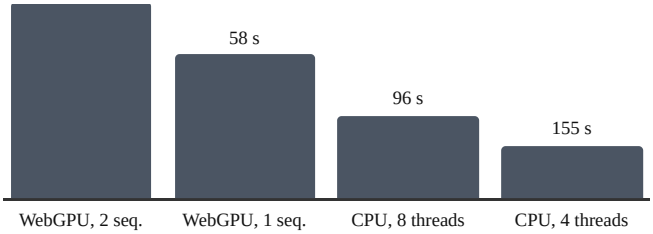


Figure 1. Time to translate the whole fixture article, by backend. The GPU path is not only faster per token; it also packs short blocks into one completion, so it issues roughly a third as many requests.

3. Scheduling

A reader does not read a page in document order, so the extension does not translate in document order either. Every block gets a priority from its distance to the viewport, its kind, and its position in the document. Blocks on screen are translated first, then blocks within about two screens in the direction the reader is scrolling. Scrolling up re-ranks the queue, which is why a reader who jumps to the conclusion gets the conclusion rather than the introduction.

Short blocks — a label, a table cell, a one-line list item — are packed into one request, up to eight per completion and 160 source tokens in total. Packing is not applied to paragraphs. A paragraph is as long as the prompt template, so packing saves almost nothing, and it delays every block in the pack behind the prefill of the whole pack. The rule is measured rather than assumed: Section 4.2 reports a run in which packing paragraphs made the median time to first text 18% worse.

Table 1. Default planning constants. The token budget is per sequence, so the two-sequence GPU path and the one-sequence CPU path share it.

Constant	Value	Meaning
maxSegmentTokens	48	Longest block that may share a completion
maxBatchTokens	160	Source tokens in one packed request
maxBatchSegments	8	Blocks in one packed request
pieceChars	600	Longest run of sentences sent as one prompt

4. Measurements

All numbers come from one laptop: eight cores, 32 GB of memory, an integrated GPU, and a browser build that supports WebGPU. The fixture is a 1,900-word article with a sidebar, a code block and a navigation bar. Every run translates the page from a cold cache, three times, and the median is reported.

4.1 Throughput

Table 2 separates the two costs a reader feels. Time to first visible paragraph is dominated by prefill and by loading the weights; full page time is dominated by decoding. The GPU path decodes 3.8 times faster than the four-thread CPU path, but it is only 2.4 times faster on the first paragraph, because the weight upload happens before either backend can produce a token.

Table 2. Median of three cold runs on the fixture article.

Backend	tok/s	First paragraph	Full page
WebGPU, 2 sequences	34.8	1.9 s	41 s
WebGPU, 1 sequence	28.1	2.1 s	58 s
CPU, 8 threads	14.6	2.8 s	96 s
CPU, 4 threads	9.1	3.4 s	155 s

4.2 What packing costs

We packed paragraphs as well as labels in an early build. The change looked harmless, since every request carried text the reader had asked for. It was not: a paragraph in a pack waited behind the prefill of every earlier paragraph, and the median time to first text rose by 18%. Restricting packing to blocks under 48 tokens recovered the loss and reduced the total number of requests by 61%.

5. Failure Modes

Three failures shaped the current design more than any feature did.

- **Reported completion over untranslated prose.** A page that skipped blocks it could not segment still said it was translated. The bar now counts blocks that never got a job and reports the page as partial instead.
- **Layout collapse in slot containers.** Writing a translation between the cells of a grid row created an anonymous item and pushed every following cell one place along. Such a container is now descended into, and each cell is translated into its own text node.
- **Stale text after a model reload.** Blocks that failed while the model was loading kept their source text forever. They are now queued again when the model reports itself loaded.

Each of these was found by a reader, not by a test, and each now has a fixture that fails without the fix. We take this as evidence that page translation is a systems problem: the model is the easy part, and the document is the hard one.

6. Remaining Work

Two shapes of document are still out of reach. The first is the PDF that a reader opens in the browser's own viewer, whose text is laid out by a PDF engine rather than by the DOM. The second is the scanned document, where the only text is a bitmap. Both need the same thing from this system: a translation that keeps the original geometry, so that a figure's caption stays under the figure and a table's column widths survive. We report on the first of these in a companion note.

Acknowledgements. The model weights are published by Preferred Networks under the PLaMo Community License. The runtime is a patched build of llama.cpp compiled with Emscripten. Neither is affiliated with this work.

References

[1] A. Author and others. “PLaMo 3 Translate: a compact translation model.” Technical report, 2026.

[2] A. Author. “llama.cpp: inference of large language models in C and C++.” Software, 2023–2026.

[3] A. Author. “wllama: WebAssembly bindings for llama.cpp.” Software, 2024–2026.

[4] World Wide Web Consortium. “WebGPU specification.” Candidate recommendation, 2025.

[5] A. Author and others. “Sequence models with learned alignments.” In a workshop on neural computation, 2017.

[6] A. Author and others. “Retrieving passages for open-domain questions.” In a workshop on neural computation, 2020.